

AWS

Amazon Web Services

DevOps Engineer

300 Interview Questions & Answers

AWS DevOps Engineer Professional
Comprehensive Interview Preparation Guide

▣ 10 Topics ▣ 300 Questions ▣ Expert Answers ▣

Akhilesh Chaudhari

TechShiksha | akhileshchaudhari.com

INDEX

AWS DevOps Engineer Professional – Topic-wise Coverage

#	Topic	Questions
1	AWS Core Services & Architecture	20
2	CI/CD & DevOps Pipelines	20
3	Containerization & Kubernetes on AWS	20
4	Infrastructure as Code & Configuration Management	15
5	Monitoring, Logging & Observability	15
6	Security, Compliance & Governance	15
7	Cost Optimization & FinOps	8
8	Networking & Connectivity	10
9	Serverless Architecture	10
10	Database & Storage Services	10
11	Advanced DevOps Practices	14
	TOTAL	157

This guide covers 10 key domains for AWS DevOps Engineer Professional exam and interviews. Each answer is written in a clear, conceptual style for easy understanding.

AWS Core Services & Architecture

Q1. What is AWS and what are its core service categories?

Ans: AWS (Amazon Web Services) is a comprehensive cloud computing platform by Amazon. It offers 200+ fully-featured services grouped into core categories: Compute (EC2, Lambda, ECS), Storage (S3, EBS, EFS), Database (RDS, DynamoDB, Aurora), Networking (VPC, Route 53, CloudFront), Security (IAM, KMS, Shield), and DevOps tools (CodePipeline, CodeBuild, CodeDeploy). AWS follows a shared responsibility model — AWS manages infrastructure security, while customers manage their data and application security.

Q2. Explain the AWS Shared Responsibility Model.

Ans: The Shared Responsibility Model divides security duties between AWS and the customer. AWS is responsible 'of the cloud' — physical datacenters, hardware, hypervisor, and managed service infrastructure. The customer is responsible 'in the cloud' — operating systems, application code, IAM configurations, encryption, and data. For example, in RDS, AWS patches the database engine, but you control who accesses the database and whether encryption is enabled.

Q3. What is an AWS Region and Availability Zone?

Ans: An AWS Region is a geographic location (e.g., us-east-1 in N. Virginia) containing multiple isolated data centers called Availability Zones (AZs). Each AZ is a separate physical facility with independent power, cooling, and networking. A region typically has 2–6 AZs. You deploy resources across multiple AZs for high availability. AZs within a region are connected via low-latency private fiber links. Example: deploying an RDS Multi-AZ instance keeps a standby in a separate AZ for automatic failover.

Q4. What is a VPC and why is it important?

Ans: A VPC (Virtual Private Cloud) is your private, logically isolated network within AWS. It gives you full control over IP address ranges (CIDR blocks), subnets, route tables, internet gateways, and security settings. Public subnets can reach the internet via an Internet Gateway; private subnets use a NAT Gateway for outbound traffic. VPCs are fundamental to AWS security — by default, resources in a VPC cannot communicate outside it. VPC Peering and AWS Transit Gateway allow VPC-to-VPC connectivity.

Q5. Explain the difference between Security Groups and NACLs.

Ans: Security Groups (SGs) are stateful, instance-level virtual firewalls — if you allow inbound traffic, the response is automatically allowed. NACLs (Network Access Control Lists) are stateless, subnet-level firewalls — you must explicitly define both inbound and outbound rules. SGs only allow rules (no explicit deny); NACLs support both allow and deny with rule numbers (processed in order). Best practice: use SGs for fine-grained instance-level control and NACLs for subnet-level broad restrictions.

Q6. What is IAM and what are its key components?

Ans: IAM (Identity and Access Management) controls who can do what in your AWS account. Key components: Users (individual identities), Groups (collection of users sharing permissions), Roles (assumed by services or external identities, no long-term credentials), Policies (JSON documents defining permissions). Best practices include least privilege principle, enabling MFA for all users, using roles for EC2 instances instead of access keys, and regularly rotating credentials.

Q7. What is the difference between IAM Roles and IAM Users?

Ans: IAM Users are long-term identities with permanent credentials (username/password or access keys), used for humans logging into AWS. IAM Roles are temporary identities assumed by AWS services, applications, or federated users — they use short-term credentials via STS (Security Token Service). For example, an EC2 instance should use an IAM Role to access S3 rather than storing access keys on the instance. Roles are more secure because credentials rotate automatically.

Q8. Explain S3 storage classes and when to use each.

Ans: S3 offers several storage classes: Standard (frequent access, low latency), Intelligent-Tiering (auto-moves objects between tiers based on access patterns), Standard-IA (infrequent access, lower cost, retrieval fee), One Zone-IA (single AZ, 20% cheaper than Standard-IA), Glacier Instant Retrieval (archive, millisecond retrieval), Glacier Flexible Retrieval (minutes to hours), Glacier Deep Archive (cheapest, 12-hour retrieval). Use lifecycle policies to automatically transition objects between classes based on age.

Q9. What is CloudFront and how does it work?

Ans: CloudFront is AWS's Content Delivery Network (CDN). It caches content at 450+ global edge locations, reducing latency for end users. When a user requests content, CloudFront serves it from the nearest edge location. If not cached, it fetches from the origin (S3, ALB, EC2, or custom HTTP server) and caches it. CloudFront supports HTTPS, WAF integration, geo-restriction, signed URLs for private content, and Lambda@Edge for running code at edge locations.

Q10. Explain the difference between EBS, EFS, and S3.

Ans: EBS (Elastic Block Store) is a block storage volume attached to a single EC2 instance — like a virtual hard drive. Best for databases and OS volumes. EFS (Elastic File System) is a managed NFS file system that can be mounted on multiple EC2 instances simultaneously — ideal for shared file storage across instances. S3 (Simple Storage Service) is object storage accessed via HTTP API — best for static assets, backups, and data lakes. EBS and EFS are AZ/region-scoped; S3 is globally accessible.

Q11. What is Auto Scaling and how does it work?

Ans: Auto Scaling automatically adjusts the number of EC2 instances (or other resources) based on demand. You define a Launch Template (AMI, instance type, SG, etc.) and an Auto Scaling Group (ASG) with minimum, desired, and maximum instance counts. Scaling policies trigger based on CloudWatch metrics (e.g., CPU > 70% → add 2 instances). Types: Simple Scaling, Step Scaling, Target Tracking (most intelligent — maintains a target metric value), and Scheduled Scaling for predictable patterns.

Q12. What is Elastic Load Balancing and what types exist?

Ans: ELB distributes incoming traffic across multiple targets. Types: ALB (Application Load Balancer) — Layer 7, HTTP/HTTPS, path/host-based routing, best for microservices; NLB (Network Load Balancer) — Layer 4, TCP/UDP, ultra-low latency, millions of requests/sec, static IP support; CLB (Classic Load Balancer) — legacy, Layer 4 and 7; GWLB (Gateway Load Balancer) — for deploying inline virtual appliances like firewalls. ALB is the most commonly used for web applications.

Q13. What is Route 53 and what routing policies does it support?

Ans: Route 53 is AWS's scalable DNS and domain registration service. Routing policies: Simple (single resource, no health checks), Weighted (split traffic by percentage — useful for A/B testing), Latency-based (routes to lowest-latency region), Failover (active-passive with health checks), Geolocation (routes based on user location), Geoproximity (routes based on geographic distance with bias), Multivalue Answer (returns multiple IPs with health checks). Route 53 integrates with CloudWatch for health monitoring.

Q14. Explain AWS Lambda and its key concepts.

Ans: Lambda is a serverless compute service — you upload code and AWS runs it without provisioning servers. Key concepts: Function (code + runtime + configuration), Trigger (event source like API Gateway, S3, SQS, or CloudWatch Events), Execution Role (IAM role granting permissions), Cold Start (initial delay when no warm container exists), Concurrency (parallel executions, default limit 1000/region), Timeout (max 15 minutes), Memory (128MB–10GB). Lambda charges per request and compute duration (millisecond billing).

Q15. What is DynamoDB and when should you use it?

Ans: DynamoDB is a fully managed, serverless NoSQL key-value and document database. Use it when you need single-digit millisecond latency at any scale, horizontal scalability, or a schema-less data model. Key concepts: Table, Item (row), Attribute (column), Primary Key (Partition Key alone, or Partition Key + Sort Key). Features: DynamoDB Streams (change events), Global Tables (multi-region replication), DAX (in-memory cache for microsecond reads), on-demand and provisioned capacity modes. Avoid for complex relational queries.

Q16. What is RDS and what databases does it support?

Ans: RDS (Relational Database Service) is a managed relational database service supporting MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, and Amazon Aurora. AWS handles backups, patching, failover, and replication. Key features: Multi-AZ (synchronous standby for HA), Read Replicas (async replication for read scaling, up to 15 for Aurora), automated backups (1–35 day retention), encryption at rest/transit, and Parameter Groups for configuration. Aurora is AWS's proprietary engine — up to 5x faster than MySQL, 3x faster than PostgreSQL.

Q17. Explain AWS CloudFormation.

Ans: CloudFormation is AWS's Infrastructure as Code (IaC) service. You define infrastructure in JSON or YAML templates, and CloudFormation provisions and manages resources as stacks. Key concepts: Template (blueprint with Resources, Parameters, Outputs, Conditions, Mappings sections), Stack (deployed instance of a template), Stack Sets (deploy across multiple accounts/regions), Change Sets (preview changes before applying), Drift Detection (identify manual changes). CloudFormation ensures consistent, repeatable infrastructure deployments.

Q18. What is AWS CloudWatch and what can it monitor?

Ans: CloudWatch is AWS's monitoring and observability service. It collects: Metrics (EC2 CPU, RDS connections, Lambda duration, etc.), Logs (application logs via CloudWatch Logs agent or Lambda), Events/EventBridge (react to AWS resource state changes), Alarms (trigger SNS/Auto Scaling based on metric thresholds). Features: Log Insights (query logs with SQL-like syntax), Container Insights, Application Insights, Contributor Insights. Custom metrics can be published via the PutMetricData API. Dashboards provide visual monitoring.

Q19. What is SNS and SQS and how are they different?

Ans: SNS (Simple Notification Service) is a pub/sub messaging service — a publisher sends a message to a topic, and all subscribers (Lambda, SQS, HTTP, email, SMS) receive it simultaneously. Good for fan-out patterns. SQS (Simple Queue Service) is a message queue — producers send messages to a queue, and consumers poll and process them one at a time. Good for decoupling services and handling load spikes. They complement each other: SNS fans out to multiple SQS queues, each processed by different consumers.

Q20. What is AWS ECS and how does it differ from EKS?

Ans: ECS (Elastic Container Service) is AWS's managed container orchestration service using task definitions and services to run Docker containers. It's simpler and tightly integrated with AWS services. EKS (Elastic Kubernetes Service) is a managed Kubernetes service — ideal if you're already using Kubernetes or need its ecosystem. ECS uses AWS-proprietary concepts; EKS uses standard Kubernetes APIs, making it more portable. Both can use Fargate (serverless compute for containers, eliminating server management) or EC2 for compute.

CI/CD & DevOps Pipelines

Q21. What is CI/CD and why is it important?

Ans: CI/CD stands for Continuous Integration and Continuous Delivery/Deployment. CI is the practice of automatically building and testing code whenever developers push changes, catching bugs early. CD automates delivery to staging or production environments. Benefits: faster release cycles, reduced human error, consistent builds, immediate feedback on code quality, and reduced integration conflicts. AWS provides CodePipeline (orchestration), CodeBuild (build/test), and CodeDeploy (deployment) as native CI/CD services.

Q22. Explain AWS CodePipeline architecture.

Ans: CodePipeline is a fully managed CI/CD orchestration service. A pipeline consists of stages (Source, Build, Test, Deploy), each containing actions. Source stage: monitors CodeCommit, GitHub, S3, or ECR for changes. Build stage: triggers CodeBuild to compile and test. Deploy stage: uses CodeDeploy, CloudFormation, ECS, or Elastic Beanstalk. Pipelines are defined in JSON and support parallel actions, manual approval gates, and cross-region deployments. Artifacts are passed between stages via S3. CodePipeline charges per active pipeline per month.

Q23. What is AWS CodeBuild and how do you configure it?

Ans: CodeBuild is a fully managed build service that compiles code, runs tests, and produces deployable artifacts. Configuration is via `buildspec.yml` at the repo root. It has phases: `install` (install dependencies), `pre_build` (login to ECR, set variables), `build` (run `mvn` package, `docker build`), `post_build` (`docker push`, create deployment package). Build environments are Docker containers — AWS provides curated images for Java, Python, Node.js, etc., or you can use custom images. Results include build logs in CloudWatch and artifacts in S3.

Q24. What deployment strategies does CodeDeploy support?

Ans: CodeDeploy supports: In-Place (stop app on each instance, deploy new version, restart — simple but causes downtime), Blue/Green for EC2 (launch new instances with new version, shift traffic, terminate old), Blue/Green for Lambda (shift traffic between function versions using Linear or Canary), Blue/Green for ECS (replace task set with new task definition). Deployment configurations control rollout speed: `AllAtOnce`, `HalfAtATime`, `OneAtATime`, or custom (e.g., `Canary10Percent5Minutes` shifts 10% of traffic first, then 100% after 5 minutes).

Q25. What is a Blue/Green deployment?

Ans: Blue/Green deployment maintains two identical production environments — Blue (current live) and Green (new version). Traffic shifts from Blue to Green after validation. Benefits: zero downtime, instant rollback (switch back to Blue), easy validation before full release. AWS implements it via: CodeDeploy (EC2/ECS/Lambda), Elastic Beanstalk environment swap, Route 53 weighted routing, or ALB target group switching. The key trade-off is cost — you temporarily run double the infrastructure.

Q26. Explain Canary deployments.

Ans: Canary deployment gradually shifts a small percentage of traffic to the new version, monitors it, then increases traffic if no issues arise. Example: 5% → 25% → 50% → 100% over defined intervals. AWS CodeDeploy Lambda deployment configurations like `Canary10Percent5Minutes` automate this. Canary releases reduce blast radius — if the new version has bugs, only a small fraction of users are affected. CloudWatch alarms can automatically trigger rollback if error rates or latency increase during the canary phase.

Q27. What is GitOps and how does it relate to AWS?

Ans: GitOps is a DevOps practice where Git is the single source of truth for infrastructure and application configuration. All changes are made via pull requests, reviewed, and merged — triggering automated deployments. On AWS, GitOps is implemented using CodePipeline or Jenkins triggered by CodeCommit/GitHub webhooks, deploying via CloudFormation, Helm charts on EKS, or Terraform. Tools like ArgoCD or Flux can sync Kubernetes state with Git. GitOps provides auditability, rollback via git revert, and collaboration.

Q28. How do you implement multi-environment deployments on AWS?

Ans: Multi-environment (dev/staging/prod) deployments use: separate AWS accounts (best for strict isolation, using AWS Organizations), or separate VPCs/namespaces within one account. CodePipeline implements promotion gates — code moves from dev to staging to prod with automated tests and manual approvals at each gate. Use Parameter Store or environment-specific CloudFormation stacks for environment-specific configurations. CodeDeploy deployment groups separate environments. Tagging resources with `Environment:dev/staging/prod` enables cost tracking and access control.

Q29. What is Infrastructure as Code (IaC) and what tools does AWS support?

Ans: IaC treats infrastructure configuration as code — version-controlled, peer-reviewed, automated. Benefits: consistency, repeatability, documentation as code, drift prevention. AWS native tool: CloudFormation (JSON/YAML). AWS CDK (Cloud Development Kit) lets you define infrastructure using TypeScript, Python, or Java, which synthesizes to CloudFormation. AWS also supports Terraform (third-party, multi-cloud) and Pulumi. Best practice: keep IaC in Git, use separate stacks per environment, and apply least-privilege IAM for deployment pipelines.

Q30. Explain AWS Elastic Beanstalk.

Ans: Elastic Beanstalk is a PaaS that abstracts infrastructure management. You upload application code, and Beanstalk automatically provisions EC2, ALB, Auto Scaling, and RDS. Supports Java, .NET, PHP, Node.js, Python, Ruby, Go, and Docker. You retain full control — Beanstalk is free, you pay only for underlying resources. Deployment options: All at Once (downtime), Rolling (partial downtime), Rolling with Additional Batch (no capacity reduction), Immutable (new ASG, zero downtime), Blue/Green (environment swap). Great for startups and teams without dedicated DevOps engineers.

Q31. What is AWS CodeArtifact?

Ans: CodeArtifact is a fully managed artifact repository for software packages — supports npm, PyPI, Maven, NuGet, Swift, and generic packages. It acts as a proxy for public repositories (npmjs.com, PyPI, Maven Central), caching packages internally for reliability and control. Teams use CodeArtifact to: centralize package management, enforce approved packages, share private packages across teams, and reduce dependency on public registries. Integrates with CodeBuild for build-time package resolution. Access is controlled via IAM and resource policies.

Q32. How does AWS CodeCommit compare to GitHub?

Ans: CodeCommit is AWS's fully managed Git repository service — private repos with IAM-based authentication, no third-party dependencies, and tight AWS service integration. Data is encrypted at rest with KMS. GitHub is the industry-standard platform with a broader ecosystem, better collaboration features (PRs, code review, Actions CI/CD), and larger community. Most organizations choose GitHub for developer experience and CodePipeline + GitHub integration. CodeCommit is preferred when data residency, compliance, or strict AWS-only tooling is required.

Q33. What are GitHub Actions and how can they deploy to AWS?

Ans: GitHub Actions is GitHub's CI/CD platform — workflows defined in YAML files under `.github/workflows/` trigger on push, PR, or schedule. To deploy to AWS: use `configure-aws-credentials` action with IAM role via OIDC (OpenID Connect — no long-term keys), then use AWS CLI or CDK deploy commands. Example steps: checkout code □ configure AWS credentials □ build Docker image □ push to ECR □ deploy to ECS via `ecs-deploy-task-definition` action. GitHub Actions integrates with all AWS services and is widely used for AWS deployments.

Q34. What is Jenkins and how is it used with AWS?

Ans: Jenkins is an open-source CI/CD automation server. On AWS, it's deployed on EC2 or EKS. Jenkins Pipelines (Jenkinsfile) define stages: Checkout, Build, Test, Docker Build, ECR Push, Deploy. AWS credentials are managed via IAM roles (EC2 instance profile) or Jenkins credentials store. Jenkins plugins include: AWS Steps, ECR, ECS Deploy, CloudFormation, Kubernetes (for EKS). While powerful and flexible, Jenkins requires self-management (patching, scaling). Many teams migrate from Jenkins to CodePipeline or GitHub Actions to reduce operational overhead.

Q35. Explain the concept of artifact management in CI/CD.

Ans: Artifact management involves storing, versioning, and distributing build outputs (JARs, Docker images, Lambda ZIPs, npm packages). In AWS CI/CD: CodeBuild stores artifacts in S3 with versioning; Docker images go to ECR (Elastic Container Registry); packages go to CodeArtifact. Best practices: use semantic versioning or commit SHA for image tags (never reuse 'latest' in production), enable artifact lifecycle policies to delete old versions, sign artifacts for integrity verification, and scan Docker images with ECR image scanning or tools like Trivy.

Q36. What is a Deployment Pipeline and what stages should it have?

Ans: A deployment pipeline automates the path from code commit to production. Essential stages: Source (code checkout, triggers pipeline), Build (compile, unit tests, lint, SAST scanning), Test (integration tests, acceptance tests), Staging Deploy (deploy to staging, smoke tests), Manual Approval (optional human gate for production), Production Deploy (blue/green or canary), Post-Deploy Verification (health checks, synthetic monitoring). Each stage should fail fast — never promote artifacts that fail earlier stages. The pipeline should complete in under 30 minutes for fast feedback.

Q37. How do you handle secrets in a CI/CD pipeline?

Ans: Never hardcode secrets in code or `buildspec.yml`. Best practices: AWS Secrets Manager (auto-rotates secrets, fine-grained access) or Parameter Store (SSM) with `SecureString` type (encrypted with KMS). In CodeBuild, reference secrets via environment variable syntax: `{{resolve:secretsmanager:MySecret:SecretString:password}}`. For GitHub Actions, use GitHub Secrets + AWS OIDC to avoid storing AWS keys. Use separate secrets per environment. Audit secret access via CloudTrail. Implement secret scanning in CI to catch accidental commits of credentials.

Q38. What is a rollback strategy in deployments?

Ans: A rollback returns the system to the last known good state when a deployment fails. Strategies: Automated Rollback (CodeDeploy triggers rollback if CloudWatch alarm breaches during deployment), Manual Rollback (redploy previous version), Blue/Green Rollback (switch traffic back to Blue environment instantly), Git Revert (revert commit and redeploy). Best practice: define rollback criteria upfront (error rate threshold, latency, health check failures), test rollback procedures regularly, maintain at least 2 previous deployment artifacts, and practice rollbacks in non-production environments.

Q39. What is AWS Proton?

Ans: AWS Proton is a managed deployment service for container and serverless applications, targeting platform engineering teams. Platform teams create environment and service templates (CloudFormation or Terraform) that developers self-service to deploy standardized infrastructure. Templates enforce organizational standards for networking, security, monitoring, and deployment. Developers don't need to know CloudFormation — they just select a template and deploy. Proton manages template versioning and propagates updates to services when templates change. Ideal for large organizations standardizing infrastructure across teams.

Q40. Explain the concept of shift-left testing.

Ans: Shift-left testing moves quality assurance earlier in the development process — 'shifting' tests left on the timeline toward development. Instead of testing only before deployment, tests run at every stage: unit tests on commit, integration tests in CI, security scans in build phase, performance tests in staging. Benefits: bugs found earlier are cheaper to fix, faster feedback loops, higher code quality. AWS CodeBuild integrates with SonarQube, Checkmarx, and OWASP ZAP for shift-left security testing. Tools like AWS CodeGuru Reviewer provide AI-powered code reviews automatically in PRs.

Containerization & Kubernetes on AWS

Q41. What is Docker and how does it work?

Ans: Docker is a containerization platform that packages applications with all dependencies into portable, isolated containers. A Dockerfile defines the image build steps. `docker build` creates an image layer by layer; `docker run` starts a container from an image. Containers share the host OS kernel but are isolated via Linux namespaces and cgroups. Key concepts: Image (read-only template), Container (running instance), Registry (image storage — ECR or Docker Hub), Volume (persistent storage), Network (bridge, host, overlay). Containers start in milliseconds vs minutes for VMs.

Q42. Explain Docker multi-stage builds.

Ans: Multi-stage builds use multiple `FROM` statements in one Dockerfile to produce lean final images. Example: Stage 1 (builder) uses a full JDK image to compile the Java app, producing a JAR. Stage 2 (runner) uses a minimal JRE image and copies only the JAR from Stage 1. The final image excludes build tools, compilers, and source code — drastically reducing image size (e.g., from 800MB to 120MB) and attack surface. Multi-stage builds are a best practice for production Docker images. Use `COPY --from=builder` to transfer artifacts between stages.

Q43. What is Amazon ECR and how do you use it?

Ans: ECR (Elastic Container Registry) is AWS's fully managed Docker container registry. It integrates with IAM for authentication — you log in via `aws ecr get-login-password` command. Repositories are private by default; public repos exist in ECR Public Gallery. Key features: image scanning (basic or enhanced with Inspector), lifecycle policies (auto-delete old images), image tag immutability (prevent tag overwrites), replication (cross-region or cross-account), and encryption at rest with KMS. Use ECR to store images for ECS, EKS, and Lambda container deployments.

Q44. What is Kubernetes and its core architecture?

Ans: Kubernetes (K8s) is an open-source container orchestration platform. Architecture: Control Plane (API Server — REST interface; etcd — distributed key-value store for cluster state; Scheduler — assigns pods to nodes; Controller Manager — runs controllers for deployments, services, etc.) and Worker Nodes (kubelet — node agent; kube-proxy — networking; container runtime — containerd/Docker). Key objects: Pod (smallest unit, 1+ containers), Deployment (manages replica sets), Service (stable network endpoint), ConfigMap, Secret, PersistentVolume.

Q45. What is EKS and how do you set it up?

Ans: EKS (Elastic Kubernetes Service) is AWS's managed Kubernetes control plane — AWS manages the API server, etcd, and control plane HA across 3 AZs. You manage worker nodes via Managed Node Groups (EC2 ASG managed by AWS), Self-Managed Node Groups, or Fargate profiles (serverless). Setup: create cluster with `eksctl` or Terraform, configure `kubectl` with `aws eks update-kubeconfig`, create node groups, install add-ons (CoreDNS, kube-proxy, VPC CNI, AWS Load Balancer Controller). Integrate with IAM via IRSA (IAM Roles for Service Accounts) for pod-level AWS permissions.

Q46. Explain Kubernetes Deployments and Rolling Updates.

Ans: A Deployment manages a ReplicaSet, ensuring the desired number of pod replicas run at all times. Rolling Update strategy gradually replaces old pods with new ones — controlled by `maxSurge` (extra pods during update) and `maxUnavailable` (pods that can be down). Example: with `maxSurge:1` and `maxUnavailable:0`, Kubernetes adds 1 new pod, waits for it to be ready, then removes 1 old pod, repeating until complete. This ensures zero downtime. `kubectl rollout undo` reverts to the previous revision. `kubectl rollout status` monitors update progress.

Q47. What is a Kubernetes Service and what types exist?

Ans: A Service provides a stable network endpoint to a set of pods selected by labels. Types: ClusterIP (internal cluster access only — default), NodePort (exposes on each node's IP at a static port, 30000-32767), LoadBalancer (provisions a cloud load balancer — on EKS, creates an ALB or NLB via AWS Load Balancer Controller), ExternalName (maps to external DNS). Services use label selectors to route traffic to matching pods. Endpoints are updated automatically as pods are added/removed. Services provide stable DNS names in the cluster (e.g., my-service.namespace.svc.cluster.local).

Q48. What is a Kubernetes Ingress?

Ans: Ingress is a Kubernetes API object that manages HTTP/HTTPS routing from outside the cluster to services inside — based on host names and URL paths. It requires an Ingress Controller to implement the rules. On EKS, the AWS Load Balancer Controller creates an ALB for each Ingress resource, or shares one ALB across multiple Ingresses using IngressGroups. An Ingress can route /api to service-A and / to service-B, or api.example.com to service-A and app.example.com to service-B. Supports TLS termination using ACM certificates on EKS.

Q49. What is Helm and how is it used?

Ans: Helm is the package manager for Kubernetes — it packages Kubernetes manifests into Charts (versioned, shareable packages). A Chart contains templates with variable substitution (values.yaml). Commands: helm install (deploy a release), helm upgrade (update), helm rollback, helm list, helm uninstall. Helm repositories (like Artifact Hub) host community charts for common software (Prometheus, nginx, cert-manager). In CI/CD pipelines, helm upgrade --install deploys to Kubernetes with environment-specific values files. Helm 3 removed Tiller for improved security.

Q50. What is IRSA in EKS?

Ans: IRSA (IAM Roles for Service Accounts) allows Kubernetes pods to assume AWS IAM roles without storing credentials. How it works: EKS supports OIDC (OpenID Connect) — you create an IAM Role with a trust policy that allows the EKS OIDC provider to assume it, scoped to a specific Kubernetes service account. Annotate the service account with the IAM role ARN. Pods using that service account automatically receive temporary AWS credentials via the AWS SDK. This follows least-privilege — each pod gets only the permissions it needs, not the full node IAM role.

Q51. What is a Kubernetes ConfigMap and Secret?

Ans: ConfigMap stores non-sensitive configuration data as key-value pairs — injected into pods as environment variables or mounted as files. Secrets store sensitive data (passwords, tokens, keys) — base64 encoded (not encrypted by default; use AWS KMS encryption with Envelope Encryption or Sealed Secrets for true encryption). Best practice: use AWS Secrets Manager or Parameter Store and the AWS Secrets and Configuration Provider (ASCP) for EKS to sync secrets from AWS to Kubernetes Secrets, ensuring centralized management and automatic rotation.

Q52. Explain Kubernetes resource requests and limits.

Ans: Resource requests are the minimum CPU/memory a container is guaranteed — used by the scheduler to place pods on nodes with sufficient capacity. Limits are the maximum CPU/memory a container can use — exceeding CPU limit causes throttling; exceeding memory limit causes OOMKilled (pod restart). Example: requests: {cpu: 100m, memory: 128Mi}, limits: {cpu: 500m, memory: 512Mi}. Setting appropriate requests/limits is critical for cluster efficiency and stability. LimitRanges enforce default limits in a namespace; ResourceQuotas limit total resource consumption per namespace.

Q53. What is the Kubernetes Horizontal Pod Autoscaler (HPA)?

Ans: HPA automatically scales the number of pod replicas based on observed CPU utilization, memory, or custom metrics (via Custom Metrics API). It checks metrics every 15 seconds and calculates desired replicas: $\text{desiredReplicas} = \text{currentReplicas} * (\text{currentMetricValue} / \text{targetMetricValue})$. You define min/max replicas to prevent over/under scaling. For EKS, KEDA (Kubernetes Event Driven Autoscaler) extends HPA to scale based on SQS queue depth, Kinesis shards, or custom AWS metrics. Cluster Autoscaler or Karpenter scales underlying EC2 nodes when pods are Pending.

Q54. What is AWS Fargate?

Ans: Fargate is serverless compute for containers — you run ECS tasks or EKS pods without managing EC2 instances. AWS handles OS patching, capacity provisioning, and node management. You specify CPU and memory per task; Fargate provisions appropriate compute. Benefits: no node management, per-second billing, strong task isolation (each task gets its own kernel), and automatic scaling. Fargate is ideal for variable workloads, batch jobs, and teams without infrastructure expertise. Trade-offs: higher per-unit cost than EC2, no GPU support, and some K8s features unavailable in Fargate.

Q55. What is Amazon ECS and how do tasks work?

Ans: ECS (Elastic Container Service) is AWS's container orchestration service. Key concepts: Task Definition (blueprint — Docker image, CPU/memory, environment variables, volumes, IAM task role), Task (running instance of a task definition), Service (ensures N tasks run continuously, integrates with ALB for traffic distribution), Cluster (logical grouping of tasks — EC2 or Fargate). ECS Services support rolling updates and blue/green deployments via CodeDeploy. Container insights provides CloudWatch monitoring. ECS is simpler than EKS and deeply integrated with AWS services like Secrets Manager, Service Discovery, and IAM.

Q56. Explain container security best practices on AWS.

Ans: Container security layers: Image Security (use minimal base images like Alpine/Distroless, scan with ECR Image Scanning or Trivy, enable tag immutability, sign images with Notary), Runtime Security (run containers as non-root user, use read-only root filesystem, drop unnecessary Linux capabilities, use Seccomp profiles), Network Security (restrict pod-to-pod traffic with Kubernetes NetworkPolicies, use VPC CNI for pod-level security groups on EKS), IAM (use IRSA for least-privilege pod permissions, never pass host IAM role to containers), and Secrets (use Secrets Manager, not environment variables for sensitive data).

Q57. What is AWS App Mesh?

Ans: App Mesh is a service mesh that provides application-level networking for microservices. It uses Envoy proxy sidecars alongside each service to control traffic flow, implement circuit breaking, retries, timeouts, and canary routing — without changing application code. Features: traffic splitting (route X% to service-v2), mutual TLS between services, distributed tracing via X-Ray, and metrics via CloudWatch. App Mesh abstracts away service discovery and resilience patterns. Use it when you have complex microservices with fine-grained traffic control needs. AWS now also supports Gateway API for EKS.

Q58. What is Karpenter?

Ans: Karpenter is an open-source node provisioner for Kubernetes, created by AWS. Unlike Cluster Autoscaler (which scales ASGs), Karpenter directly calls EC2 APIs to provision the right node for pending pods – selecting optimal instance types based on pod requirements. Benefits: faster node provisioning (30 seconds vs 3+ minutes), bin-packing (selects cheapest instance types), Spot instance support with automatic fallback, and scheduled de-provisioning. Karpenter is the recommended node autoscaler for EKS, replacing Cluster Autoscaler for most use cases.

Q59. Explain Docker networking modes.

Ans: Docker networking modes: Bridge (default – container gets its own network namespace, connected via virtual bridge docker0, NAT for outbound), Host (container shares host network namespace – no port mapping needed, better performance, less isolation), None (no network – fully isolated container), Overlay (for multi-host Docker Swarm or Kubernetes clusters – containers on different hosts communicate via VXLAN), Macvlan (container gets its own MAC address, appears as physical device on network – for legacy apps needing direct network access). On EKS, AWS VPC CNI plugin assigns VPC IP addresses directly to pods.

Q60. What is a sidecar container pattern?

Ans: The sidecar pattern runs a helper container alongside the main application container in the same pod. Sidecar containers share the same network namespace and volumes. Common uses: Service Mesh Proxy (Envoy/Istio – intercepts traffic for observability and security), Log Collection (Fluentd sidecar collects and forwards logs to CloudWatch or Elasticsearch), AWS X-Ray Daemon (collects traces from application and forwards to X-Ray), Configuration Sync (pulls config from Parameter Store and writes to shared volume), SSL/TLS Termination (nginx sidecar handles HTTPS). Sidecars add functionality without modifying the main application.

Infrastructure as Code & Configuration Management

Q61. What is Terraform and how does it work?

Ans: Terraform is an open-source IaC tool by HashiCorp. You write infrastructure configuration in HCL (HashiCorp Configuration Language), and Terraform creates an execution plan showing changes before applying. Key commands: terraform init (initialize, download providers), terraform plan (preview changes), terraform apply (create/update resources), terraform destroy (tear down). State file (terraform.tfstate) tracks deployed resources — store it in S3 with DynamoDB locking for teams. Providers are plugins for cloud APIs (aws, azure, google). Modules are reusable configurations.

Q62. What is the difference between Terraform and CloudFormation?

Ans: CloudFormation is AWS-native, deeply integrated, free, and supports all AWS services on day of launch. Terraform is multi-cloud (AWS, Azure, GCP, 1000+ providers), uses HCL (more readable than JSON/YAML), has a larger community, and supports advanced logic with loops and conditionals. CloudFormation has better AWS-specific features (StackSets, Drift Detection, Service Catalog integration). Terraform requires state management. For AWS-only organizations, CloudFormation is often preferred; for multi-cloud, Terraform is standard. AWS CDK bridges the gap — code-first with CloudFormation backend.

Q63. What is Terraform remote state and why is it important?

Ans: By default, Terraform stores state locally — problematic for teams (conflicts, loss). Remote state stores the tfstate file in a shared location. AWS setup: S3 bucket for state storage + DynamoDB table for state locking. Configure in backend block: `backend "s3" { bucket="my-tfstate", key="prod/terraform.tfstate", region="us-east-1", dynamodb_table="terraform-locks" }`. Benefits: team collaboration, state locking (prevents concurrent apply), state history (S3 versioning), secure access via IAM. Workspaces allow separate state per environment using one configuration.

Q64. Explain Terraform modules.

Ans: Modules are reusable Terraform configurations — packages of resources that can be called with different inputs. A module has inputs (variables), outputs, and resources. Example: a VPC module accepts `cidr_block` and environment variables, creates VPC, subnets, route tables, and outputs `vpc_id` and `subnet_ids`. Call it: `module "vpc" { source = "../modules/vpc", cidr_block = "10.0.0.0/16" }`. Public modules are available in the Terraform Registry (e.g., `terraform-aws-modules/vpc/aws`). Modules promote DRY (Don't Repeat Yourself) principles and organizational standards.

Q65. What is AWS CDK?

Ans: AWS CDK (Cloud Development Kit) lets you define AWS infrastructure using familiar programming languages — TypeScript, Python, Java, C#, Go. You write classes and functions; CDK synthesizes them into CloudFormation templates. Benefits: IDE autocomplete, type safety, loops and conditions (unlike YAML), reusable constructs (similar to Terraform modules), and unit testing with jest or pytest. CDK Constructs library provides high-level abstractions (e.g., `ApplicationLoadBalancedFargateService` creates ALB + ECS in one line). CDK is the modern way to write CloudFormation, recommended for teams already using programming languages.

Q66. What is Ansible and how is it used with AWS?

Ans: Ansible is an agentless configuration management and automation tool — uses SSH/WinRM to connect to servers and YAML Playbooks to define desired state. AWS modules (`amazon.aws` collection) manage EC2 instances, S3 buckets, RDS, security groups, and more. Dynamic inventory fetches running EC2 instances by tags. Ansible complements IaC: Terraform provisions infrastructure, Ansible configures servers (install packages, deploy apps, configure services). Use cases: OS hardening, application deployment, configuration drift remediation. Ansible Tower/AWX provides GUI and scheduling.

Q67. What is AWS Systems Manager (SSM)?

Ans: SSM is a unified operations hub for AWS resources — no SSH required. Key features: Session Manager (browser-based or CLI shell access to EC2 without opening port 22, full audit trail via CloudTrail), Run Command (execute scripts/commands across multiple instances), Patch Manager (automated OS patching with maintenance windows), State Manager (enforce desired configuration state), Parameter Store (centralized config and secrets storage), Inventory (collect software/hardware inventory from instances), and Automation (runbooks for common operational tasks like AMI creation). SSM Agent must be installed on EC2 instances.

Q68. What is AWS Config?

Ans: AWS Config continuously monitors and records AWS resource configurations and changes. It evaluates resources against Config Rules (managed rules like s3-bucket-public-read-prohibited or custom Lambda-backed rules). Use cases: compliance auditing (prove resources have been correctly configured), security analysis (detect misconfigured security groups), change management (who changed what and when), and troubleshooting (configuration history). Conformance Packs bundle multiple rules for compliance frameworks like CIS, PCI-DSS, or HIPAA. Config integrates with Security Hub for centralized security findings.

Q69. Explain immutable infrastructure.

Ans: Immutable infrastructure means servers are never modified after deployment — instead of patching or updating in-place, you build a new image and replace the old one. Benefits: eliminates configuration drift (servers diverging from their intended state), faster rollbacks (switch to previous AMI), consistent environments, and simpler troubleshooting. AWS implementation: build a new AMI with Packer when app changes, update the Launch Template, and perform an Auto Scaling rolling update to replace instances. All state is stored externally (RDS, ElastiCache, S3). Containers (Docker) are inherently immutable — you replace containers, not modify them.

Q70. What is AWS OpsWorks?

Ans: OpsWorks is a configuration management service supporting Chef and Puppet. OpsWorks Stacks uses layers (Load Balancer, Application, Database) with Chef recipes for automated configuration. OpsWorks for Chef Automate and OpsWorks for Puppet Enterprise are managed Chef/Puppet masters. Use OpsWorks when migrating existing Chef/Puppet workflows to AWS. For new implementations, AWS recommends Systems Manager or Ansible. OpsWorks Stacks reached end-of-life in May 2024 for new customers.

Q71. What is Packer and how is it used?

Ans: Packer (by HashiCorp) automates machine image creation — builds identical AMIs, Docker images, or VMware images from a single template. A Packer template defines: source (base AMI, region, instance type), builders (where to build), provisioners (shell scripts, Ansible playbooks, Chef cookbooks to configure the image), and post-processors (compress, push to registry). In CI/CD: Packer runs in CodeBuild, bakes the application into an AMI, tags it with build number, and stores the AMI ID in Parameter Store for Auto Scaling Group updates. This enables immutable infrastructure patterns.

Q72. Explain drift detection in IaC.

Ans: Configuration drift occurs when manually applied changes cause resources to differ from their IaC-defined state. Drift detection identifies these discrepancies. CloudFormation: Stack ▢ Detect Drift shows drifted resources and property-level differences. Terraform: terraform plan shows differences between state and actual resources. AWS Config continuously monitors and alerts on configuration changes. Best practices to prevent drift: block console access via SCPs (Service Control Policies), enforce all changes via pull requests, use Config Rules to auto-remediate drift, and run drift detection nightly with alerts to the operations team.

Q73. What is AWS Service Catalog?

Ans: Service Catalog allows organizations to create and manage approved CloudFormation templates as Products in a Portfolio. Administrators create standardized products (pre-approved EC2 setups, RDS configurations, VPC templates); end users self-service provision them without needing CloudFormation expertise. Governance: administrators control which products are available to which teams via Portfolio sharing and IAM constraints. Launch constraints allow products to be provisioned with elevated permissions. Integrates with Config for compliance. Reduces time-to-provision while maintaining security standards.

Q74. What are CloudFormation StackSets?

Ans: StackSets extend CloudFormation stacks across multiple AWS accounts and regions from a single operation. Use cases: deploy baseline security controls (CloudTrail, Config, GuardDuty) across all accounts in an AWS Organization, standardize VPC configurations, enforce tagging policies. StackSets use service-managed permissions (AWS Organizations integration) or self-managed permissions (IAM roles). Operations: CreateStackInstances, UpdateStackInstances, DeleteStackInstances — all operate across target account/region combinations. Drift detection works at StackSet level. Automatic deployment to new accounts when they join an OU.

Q75. Explain AWS Landing Zone and Control Tower.

Ans: A Landing Zone is a well-architected, multi-account AWS environment baseline — provides networking (hub-and-spoke VPC), identity (SSO), logging (centralized CloudTrail/Config), and security guardrails. AWS Control Tower automates Landing Zone setup — it creates a management account, Log Archive account, and Audit account. Account Factory (based on Service Catalog) provisions new accounts with pre-configured baselines. Guardrails are mandatory (enforced via SCPs) or strongly recommended policies. Controls library includes 500+ proactive, preventive, and detective controls aligned with AWS best practices.

Monitoring, Logging & Observability

Q76. What is Observability and how does it differ from Monitoring?

Ans: Monitoring tells you when something is wrong (alerts on known failure modes). Observability lets you understand WHY it's wrong — enabling debugging of unknown failure modes. Observability has three pillars: Metrics (quantitative measurements — CPU, request rate, error rate), Logs (timestamped records of discrete events — application errors, audit trails), and Traces (end-to-end request flow across microservices — shows latency by service). AWS provides CloudWatch (metrics/logs), X-Ray (traces), and CloudWatch Container Insights, Application Insights, and ServiceLens for integrated observability.

Q77. What is AWS X-Ray?

Ans: X-Ray provides distributed tracing for applications — it traces requests as they travel through microservices, Lambda, API Gateway, DynamoDB, and more. The X-Ray SDK instruments your application code to emit trace data. X-Ray Daemon collects and batches trace data before sending to X-Ray service. Service Map visualizes your architecture with latency and error rates per service. Trace analysis identifies bottlenecks, errors, and throttling. Sampling rules control what percentage of requests are traced (to manage cost). AWS X-Ray integrates with ECS, Lambda, EC2, API Gateway, and EKS (via ADOT Collector).

Q78. What is the ELK Stack and how is it used on AWS?

Ans: ELK Stack consists of Elasticsearch (search/analytics engine), Logstash (data ingestion pipeline), and Kibana (visualization dashboard). On AWS, Amazon OpenSearch Service (formerly Elasticsearch Service) is the managed option. Log pipeline: application → CloudWatch Logs → Lambda or Kinesis Firehose → OpenSearch → Kibana dashboards. Use Firehose for high-throughput log delivery with automatic batching and compression. OpenSearch supports full-text search, complex aggregations, and anomaly detection. Alternative: use CloudWatch Logs Insights for simpler ad-hoc log querying without the OpenSearch overhead.

Q79. What is Amazon OpenSearch Service?

Ans: OpenSearch Service is a managed, scalable search and analytics service based on OpenSearch (Apache 2.0 fork of Elasticsearch). Use cases: log analytics, full-text search, real-time application monitoring, security analytics. Features: managed cluster provisioning, automated backups, OpenSearch Dashboards (Kibana equivalent), encryption at rest/transit, VPC isolation, fine-grained access control with IAM and internal user database, and UltraWarm/Cold storage tiers for cost optimization. Ingest data via Kinesis Firehose, Lambda, or Logstash. Scale by adding nodes or enabling auto-tune for performance optimization.

Q80. What is AWS CloudTrail?

Ans: CloudTrail records all API calls made in your AWS account — who did what, when, and from where. It captures: management events (control plane operations — create/modify/delete resources), data events (S3 object access, Lambda invocations, DynamoDB item operations — high volume, optional), and Insights events (unusual API activity detection). CloudTrail logs go to S3 and optionally to CloudWatch Logs. Best practices: enable CloudTrail in all regions, enable log file integrity validation (SHA-256 hashing), restrict S3 bucket access, and set up CloudWatch alarms for suspicious activity (e.g., root login, IAM policy changes).

Q81. What are CloudWatch Logs and how do you analyze them?

Ans: CloudWatch Logs centralizes logs from EC2 (via CloudWatch Agent), Lambda (automatic), ECS, EKS, API Gateway, and more. Concepts: Log Group (application/service), Log Stream (individual instance/function). CloudWatch Logs Insights provides SQL-like query language — fields @timestamp, @message, parse, filter, stats, sort, limit. Example: filter @message like /ERROR/ | stats count(*) as errors by bin(5m). Log metric filters extract numeric values from logs and create CloudWatch metrics for alarming. Export logs to S3 for long-term storage; use Kinesis Firehose for real-time streaming to OpenSearch or S3.

Q82. What is Amazon Managed Grafana?

Ans: Amazon Managed Grafana is a fully managed Grafana service — AWS handles provisioning, patching, scaling, and high availability. Grafana is the leading open-source visualization platform for metrics and logs. Managed Grafana integrates with AWS data sources out-of-the-box: CloudWatch, X-Ray, OpenSearch, Athena, Timestream, and third-party sources like Prometheus, InfluxDB, and Datadog. Authentication via AWS SSO or SAML. Use it for unified dashboards combining AWS metrics with application metrics, creating NOC-style operational dashboards, and on-call runbook integration with PagerDuty.

Q83. What is AWS Distro for OpenTelemetry (ADOT)?

Ans: ADOT is AWS's distribution of the OpenTelemetry project — a vendor-neutral, open-source observability framework for collecting traces, metrics, and logs. ADOT Collector runs as a sidecar (ECS/EKS) or agent (EC2), receiving telemetry from applications via OTLP protocol and exporting to AWS X-Ray, CloudWatch, and Amazon Managed Prometheus. OpenTelemetry SDKs (available for Java, Python, Node.js, .NET, Go) instrument application code. ADOT enables multi-backend observability — same instrumentation exports to X-Ray AND Datadog without code changes. It's the future-proof observability standard.

Q84. What is Amazon Managed Service for Prometheus (AMP)?

Ans: AMP is a fully managed, Prometheus-compatible monitoring service for container metrics. Prometheus is the CNCF standard for Kubernetes metrics collection. AMP eliminates the need to manage Prometheus servers, storage, and HA. ADOT Collector or Prometheus with `remote_write` scrapes EKS pod metrics and sends to AMP workspace. Metrics are queried using PromQL in Grafana (Managed Grafana integrates natively with AMP). AMP retains data for 150 days, with configurable alerting rules that trigger SNS. Use AMP + Managed Grafana for the fully managed Prometheus/Grafana stack on AWS.

Q85. What is a CloudWatch Dashboard?

Ans: CloudWatch Dashboards are customizable views of metrics from multiple services and regions in a single pane. You add widgets: Line/Bar charts (metric trends), Number (current value), Alarm Status, Logs Insights results, or custom text/markdown. Dashboards support variables for filtering by account, region, or custom dimensions. Cross-account/cross-region dashboards consolidate metrics from multiple AWS accounts. Export dashboards as JSON for version control and sharing. Automatic Dashboards (pre-built by AWS) provide instant visibility for services like EC2, RDS, Lambda, and API Gateway without any configuration.

Q86. What is AWS Health Dashboard?

Ans: AWS Health Dashboard provides personalized information about AWS service events affecting your account. Personal Health Dashboard shows: events impacting your specific resources (not just general service degradation), upcoming maintenance windows, planned lifecycle events (deprecations), and security bulletins. AWS Health API allows programmatic access — use it to automate responses (e.g., EventBridge rule triggers Lambda when an EC2 host retirement event occurs, automatically stopping and restarting the instance on a new host). Integrate with Slack or PagerDuty via EventBridge for real-time notifications to on-call teams.

Q87. What are CloudWatch Alarms?

Ans: CloudWatch Alarms monitor metrics and trigger actions when thresholds are breached. States: OK, ALARM, INSUFFICIENT_DATA. Actions: SNS notifications (email, SMS, Lambda, PagerDuty), Auto Scaling policies, EC2 actions (stop, reboot, terminate, recover). Composite Alarms combine multiple alarms with AND/OR logic – reduce alert noise by alerting only when both high CPU AND high latency occur. Anomaly Detection Alarms use ML to establish baselines and alert on unusual patterns without fixed thresholds. Best practice: create alarms for SLI (Service Level Indicators) and set alarm thresholds at SLO (Service Level Objective) boundaries.

Q88. What is Kinesis and how is it used for log streaming?

Ans: Kinesis is AWS's real-time data streaming platform. Components: Kinesis Data Streams (durable, ordered stream – producers put records, consumers process in real-time, 1–365 day retention), Kinesis Firehose (delivery service – buffers and loads to S3, OpenSearch, Redshift, Splunk with transformation via Lambda), Kinesis Data Analytics (SQL or Apache Flink for real-time stream processing). Log use case: CloudWatch Logs → Kinesis subscription filter → Firehose → S3 (raw archive) + OpenSearch (real-time search). Kinesis scales by adding shards (1MB/s write, 2MB/s read per shard).

Q89. How do you implement SLIs, SLOs, and SLAs on AWS?

Ans: SLIs (Service Level Indicators) are metrics measuring service quality – availability (% successful requests), latency (p99), error rate, throughput. SLOs (Service Level Objectives) are targets for SLIs – e.g., 99.9% availability, p99 latency < 200ms. SLAs (Service Level Agreements) are contractual commitments with penalties. AWS implementation: create CloudWatch metrics for SLIs, set Alarms when SLO thresholds approach (alert at 50%, 75%, 90% of error budget consumed), create CloudWatch Dashboard with error budget burn rate, and use CloudWatch Synthetics for external availability monitoring. Error budget tracking enables data-driven decisions about feature work vs reliability work.

Q90. What is AWS CloudWatch Synthetics?

Ans: CloudWatch Synthetics allows you to create Canaries — synthetic monitoring scripts that simulate user interactions with your application. Canaries run on a schedule (every minute, hourly) and verify that your endpoints work correctly, independently of actual user traffic. Use cases: test login flows, API endpoints, broken links, SSL certificate expiration, and page load performance. If a Canary fails, it alerts CloudWatch Alarms and captures screenshots and HAR files for debugging. Visual monitoring Canaries compare screenshots against baselines to detect UI regressions. Synthetics run from multiple AWS regions for global availability monitoring.

Security, Compliance & Governance

Q91. What is the AWS Well-Architected Framework?

Ans: The Well-Architected Framework provides AWS best practices across six pillars: Operational Excellence (run and monitor systems, continuous improvement), Security (protect data, systems, assets), Reliability (ability to recover from failures, meet demand), Performance Efficiency (use resources efficiently), Cost Optimization (avoid unnecessary costs), and Sustainability (minimize environmental impact). The Well-Architected Tool in AWS console runs reviews against these pillars, identifying high-risk issues (HRIs) and generating improvement plans. Use it for new workload design and periodic reviews of existing architectures.

Q92. What is AWS GuardDuty?

Ans: GuardDuty is a threat detection service that continuously monitors CloudTrail, VPC Flow Logs, and DNS logs using ML and threat intelligence to identify malicious activity. It detects: compromised EC2 instances (communicating with known C2 servers), credential compromise (unusual API calls from unexpected locations), privilege escalation attempts, cryptomining, data exfiltration, and IAM anomalies. Findings appear in GuardDuty console and EventBridge for automated remediation (quarantine compromised instances, revoke IAM credentials). No agents required — fully managed, low-latency detection. Enable in all regions via Organizations integration.

Q93. What is AWS Security Hub?

Ans: Security Hub provides a centralized security posture management view. It aggregates findings from GuardDuty, Inspector, Macie, IAM Access Analyzer, Firewall Manager, and third-party tools (CrowdStrike, Palo Alto) into a normalized format. Security Hub runs automated security checks against CIS AWS Foundations Benchmark, PCI-DSS, and AWS Foundational Security Best Practices. Findings are prioritized by severity. Integrates with EventBridge for automated remediation workflows. In multi-account Organizations setups, a designated Security Hub administrator account aggregates findings from all member accounts.

Q94. What is AWS Inspector?

Ans: Inspector is an automated vulnerability management service. Inspector v2 continuously scans: EC2 instances (OS packages and software vulnerabilities using CVE database), ECR container images (on push and continuously for new CVEs), and Lambda functions (code vulnerabilities and outdated packages). It prioritizes findings by exploitability and environmental context, not just CVSS score. Integrates with Security Hub and EventBridge for automated patching workflows. Enable once — Inspector automatically discovers and scans new resources as they're launched. Replace legacy periodic scans with continuous, automated vulnerability detection.

Q95. What is AWS Macie?

Ans: Macie is a data security service that uses ML to automatically discover, classify, and protect sensitive data in S3. It identifies PII (personally identifiable information — names, addresses, SSNs, credit card numbers), financial data, and credentials in S3 buckets. Macie also identifies security risks: publicly accessible buckets, unencrypted buckets, cross-account access. Findings sent to Security Hub and EventBridge. Use cases: GDPR/HIPAA compliance, data lake security, breach prevention. Macie samples data (doesn't scan every byte) and assigns confidence scores. Schedule sensitive data discovery jobs or run on-demand.

Q96. What is AWS WAF?

Ans: WAF (Web Application Firewall) protects web applications from common exploits – SQL injection, cross-site scripting (XSS), HTTP floods, bot traffic, and OWASP Top 10 vulnerabilities. Deploy WAF on ALB, CloudFront, API Gateway, or AppSync. Rules: IP-based (block specific IPs/ranges), Rate-based (limit requests per IP per 5 minutes), Managed Rule Groups (AWS or Marketplace – pre-built rules for OWASP, Bot Control, Account Takeover Prevention, Fraud Control). WAF Logs go to S3, CloudWatch Logs, or Kinesis Firehose for analysis. Use Shield Advanced for DDoS protection combined with WAF.

Q97. What is AWS Shield?

Ans: Shield provides DDoS protection. Shield Standard is free, automatic, protects all AWS resources against common Layer 3/4 attacks (SYN floods, UDP reflection). Shield Advanced (paid – \$3,000/month + data transfer) adds: Layer 7 DDoS protection (HTTP floods), real-time attack visibility, DDoS cost protection (no surge charges during attacks), access to AWS DDoS Response Team (DRT), and WAF integration for automatic rule creation during attacks. Protects EC2 EIPs, ELB, CloudFront, Route 53, and Global Accelerator. Use Shield Advanced for mission-critical, revenue-generating applications.

Q98. What is AWS KMS and how does it work?

Ans: KMS (Key Management Service) is a managed service for creating and controlling cryptographic keys. KMS Keys (formerly CMKs): AWS Managed (created and rotated by AWS for each service), Customer Managed (you create, control, and optionally rotate annually), and External (import your own key material). Envelope Encryption: KMS generates a Data Encryption Key (DEK), encrypts data with the DEK, then encrypts the DEK with the KMS key – only the encrypted DEK is stored. Services using KMS: S3, EBS, RDS, DynamoDB, Secrets Manager, CloudTrail. All KMS API calls logged in CloudTrail. Key policies and grants control who can use keys.

Q99. What is AWS Secrets Manager?

Ans: Secrets Manager stores, manages, and rotates secrets — database passwords, API keys, OAuth tokens. Features: automatic rotation (built-in Lambda functions for RDS, Redshift, DocumentDB, or custom Lambda for any secret), versioning (current and previous versions available during rotation), fine-grained IAM policies, cross-account access via resource policies, and replication to multiple regions for disaster recovery. Applications retrieve secrets via SDK or CLI — no hardcoded credentials. Integration: ECS Task Definitions, Lambda environment variables, CodeBuild environment variables, and Kubernetes via ASCP all support native Secrets Manager integration.

Q100. What is AWS Organizations and SCPs?

Ans: Organizations lets you manage multiple AWS accounts centrally. Hierarchy: Management Account □ Organizational Units (OUs — e.g., Production, Sandbox, Security) □ Member Accounts. Service Control Policies (SCPs) are permission guardrails applied to OUs or accounts — they restrict what actions IAM users and roles can perform, even if they have permissive IAM policies. Example SCP: deny all actions outside us-east-1 (data residency), deny S3 bucket public access, require MFA for IAM user changes. SCPs don't grant permissions — they only restrict. Use with Control Tower Guardrails for comprehensive governance.

Q101. What is IAM Access Analyzer?

Ans: IAM Access Analyzer identifies resources shared with external principals — finding unintended public or cross-account access. It analyzes resource-based policies on S3 buckets, IAM roles, KMS keys, Lambda functions, SQS queues, and Secrets Manager secrets. Findings show the external principal, access level, and source (internet, specific account, organization). Policy Validation checks IAM policies against best practices and suggests improvements. Unused access analyzer identifies IAM roles, users, and permissions not used in the last 90 days — helps right-size permissions. Archive findings once reviewed and confirmed intentional.

Q102. What is AWS Certificate Manager (ACM)?

Ans: ACM provisions, manages, and renews SSL/TLS certificates for AWS services. It offers free public certificates for domains you control — validated via DNS (recommended, auto-renewals with Route 53) or email. Certificates are deployed to ALB, CloudFront, API Gateway, and NLB. ACM Private CA issues private certificates for internal services, mTLS, and code signing. Key benefits: automatic renewal (ACM renews certificates 60 days before expiration), no certificate private key management (private keys are never accessible — ACM manages them), and integration with Route 53 for DNS validation in seconds.

Q103. What is AWS Firewall Manager?

Ans: Firewall Manager is a centralized security management service for multi-account, multi-region environments. It enforces WAF rules, Shield Advanced protections, Security Groups, Network Firewalls, and Route 53 Resolver DNS Firewall policies across all accounts in an Organization. When new accounts join or new resources are created, Firewall Manager automatically applies security policies. Audit policies identify non-compliant resources. Use cases: ensure all ALBs have WAF, ensure all EC2 instances have compliant security groups, enforce VPC network firewall rules. Requires Security Hub and Organizations.

Q104. What is AWS Network Firewall?

Ans: Network Firewall is a managed, stateful network firewall for VPC traffic inspection. It provides: stateful deep packet inspection (Layer 7), domain list filtering (block access to malicious domains), intrusion detection and prevention (Suricata-compatible rules), and TLS inspection. Deploy in a dedicated inspection VPC — route traffic through the firewall using Gateway Load Balancer or centralized architecture. Use cases: east-west traffic inspection between VPCs, egress filtering (prevent data exfiltration), compliance requirements for network-level controls. Pairs with WAF (Layer 7 HTTP), Shield (DDoS), and GuardDuty for layered defense.

Q105. What is compliance as code?

Ans: Compliance as code encodes regulatory requirements (SOC 2, HIPAA, PCI-DSS, GDPR) as automated checks that run continuously. AWS implementation: Config Rules (detect non-compliant resources), Config Conformance Packs (bundle rules for compliance frameworks), Security Hub Standards (automated checks against CIS, PCI-DSS), and SCPs (preventive controls). Infrastructure is checked in CI/CD pipelines using tools like cfn-nag (CloudFormation security linting), Checkov (IaC scanning for Terraform/CloudFormation), and tfsec. Non-compliant resources trigger EventBridge rules for automatic remediation or ServiceNow tickets for manual review. Compliance dashboards in Security Hub show posture over time.

Cost Optimization & FinOps

Q106. What is AWS Cost Explorer?

Ans: Cost Explorer is a cost analysis tool — visualize, understand, and manage AWS spending. Features: 13 months of historical data, hourly/daily/monthly granularity, filtering by service/tag/region/account, Savings Plans and Reserved Instance coverage reports, and rightsizing recommendations. RI Coverage and SP Utilization dashboards identify unused commitments. Cost Explorer API enables programmatic cost analysis. Use with Cost Allocation Tags (activate tags like Environment, Team, Project) to chargeback costs to business units. Forecasting predicts future costs based on historical patterns. Cost Anomaly Detection uses ML to alert on unexpected spending.

Q107. What are AWS Savings Plans?

Ans: Savings Plans provide up to 66% savings over On-Demand pricing in exchange for a commitment to consistent usage (measured in \$/hour) for 1 or 3 years. Types: Compute Savings Plans (most flexible — applies to any EC2 instance, Lambda, Fargate, regardless of region, family, OS), EC2 Instance Savings Plans (highest discount — specific instance family in a region), SageMaker Savings Plans. Unlike Reserved Instances, Savings Plans automatically apply as you change instance types or migrate workloads to Fargate. Recommendation: start with Compute Savings Plans to cover baseline, not peak usage.

Q108. What are EC2 Spot Instances?

Ans: Spot Instances use spare EC2 capacity at discounts up to 90% off On-Demand. Trade-off: AWS can reclaim Spot Instances with a 2-minute interruption notice when capacity is needed. Best use cases: batch processing, big data analytics, CI/CD build servers, stateless web servers, and machine learning training. Mitigation strategies: Spot Instance diversification (request multiple instance types/AZs), use Spot Interruption Notices to gracefully drain work, design fault-tolerant applications with checkpointing, use EC2 Fleet or Auto Scaling with On-Demand base capacity + Spot surge capacity. EKS Karpenter handles Spot interruptions automatically.

Q109. What is AWS Trusted Advisor?

Ans: Trusted Advisor is an automated best practices recommendation service. It checks your AWS account across five categories: Cost Optimization (idle EC2, unused EIPs, underutilized RDS), Security (open security groups, MFA on root, S3 bucket permissions), Fault Tolerance (no Multi-AZ RDS, insufficient Auto Scaling capacity), Performance (high EC2 utilization, CloudFront misconfiguration), and Service Limits (approaching AWS service quotas). Basic checks are free; Business and Enterprise Support plans unlock all checks and Trusted Advisor API. Set up weekly Trusted Advisor notifications via email.

Q110. What is AWS Budgets?

Ans: AWS Budgets lets you set custom cost and usage budgets with alerts. Budget types: Cost (total spend), Usage (service-specific units — EC2 hours, S3 GB), RI/SP Utilization, RI/SP Coverage. Alerts: Actual (when spend exceeds threshold) or Forecasted (when predicted to exceed). Actions: automatically apply IAM policies, SCP, or run SSM Automation when budget thresholds are reached (e.g., restrict EC2 launches when budget is 80% consumed). Budgets Reports schedule daily/weekly/monthly email summaries. Use budget filters (service, region, tag, account) for granular tracking. Ideal for chargeback and showback reporting.

Q111. What is rightsizing in AWS?

Ans: Rightsizing matches instance size to workload requirements — eliminating over-provisioned resources that waste money. Process: collect CloudWatch metrics (CPU, memory, network), identify instances with consistently low utilization (CPU < 20% for 14+ days), recommend downsizing (m5.xlarge → m5.large saves 50%). AWS Compute Optimizer uses ML to analyze CloudWatch metrics and recommend optimal EC2, Auto Scaling, Lambda, ECS, and EBS configurations. Tools: Cost Explorer Rightsizing Recommendations, Trusted Advisor, third-party tools (CloudHealth, Apptio Cloudability). Establish rightsizing as a monthly FinOps review process.

Q112. Explain AWS Reserved Instances.

Ans: Reserved Instances (RIs) provide up to 72% discount over On-Demand for EC2, RDS, ElastiCache, Redshift, and OpenSearch in exchange for a 1 or 3-year commitment. Payment options: All Upfront (highest discount), Partial Upfront, No Upfront (lowest discount). Types: Standard RIs (highest discount, specific instance family/region/OS), Convertible RIs (lower discount but can change instance family, OS, tenancy — more flexibility). RI Marketplace allows selling unused Standard RIs. RIs apply automatically to matching usage. Prefer Savings Plans over EC2 Standard RIs for flexibility, but use RDS RIs for database cost optimization.

Q113. What is AWS Cost Anomaly Detection?

Ans: Cost Anomaly Detection uses ML to continuously monitor your AWS costs and identify unusual spending. It learns your spending patterns and alerts when costs deviate significantly — detecting compromised credentials causing unexpected EC2 launches, misconfigured Auto Scaling, runaway data transfer, or sudden Spot price spikes. Configure monitors by AWS Service, Member Account, Cost Allocation Tag, or Cost Category. Set alert thresholds (e.g., alert if anomaly > \$100 or 30% above expected). Alerts via SNS/email include root cause analysis identifying the service and usage type driving the anomaly.

Networking & Connectivity

Q114. What is AWS Transit Gateway?

Ans: Transit Gateway is a regional network hub that connects VPCs and on-premises networks. Instead of VPC peering ($N*(N-1)/2$ connections for N VPCs – complex mesh), Transit Gateway creates a hub-and-spoke topology – all VPCs connect to the TGW. Supports: VPC attachments, VPN connections, Direct Connect Gateways, SD-WAN (via third-party appliances), and inter-region peering between TGWs. Route Tables on TGW control VPC-to-VPC connectivity (isolate prod from dev). Multicast support for media streaming. Bandwidth up to 50 Gbps per VPC attachment. Simplifies network management at enterprise scale.

Q115. What is AWS Direct Connect?

Ans: Direct Connect provides a dedicated, private physical network connection between on-premises and AWS – bypassing the public internet. Benefits: consistent bandwidth, lower latency, reduced data transfer costs, and improved security for sensitive data. Connection types: Dedicated (1Gbps, 10Gbps, 100Gbps physical ports at Direct Connect locations), Hosted (50Mbps–10Gbps via APN partners). Use with Virtual Private Gateway (VPG) or Transit Gateway for VPC access. Direct Connect + VPN provides encrypted, redundant connectivity. Implement redundant connections across separate Direct Connect locations for HA (per AWS recommendation).

Q116. What is VPC Peering?

Ans: VPC Peering creates a direct network connection between two VPCs – traffic uses the AWS backbone, never transits the internet. Peered VPCs can communicate using private IPs as if in the same network, but CIDRs must not overlap. VPC Peering is non-transitive – if VPC A peers with B, and B peers with C, A cannot reach C via B. Limitations: no bandwidth limits (scales with instance type), no transitive routing, no edge-to-edge routing (can't use VPN or Direct Connect from peered VPC). Use Transit Gateway for complex multi-VPC topologies. Peering works across accounts and regions (inter-region peering).

Q117. What is AWS PrivateLink?

Ans: PrivateLink provides private connectivity to AWS services, third-party services, and your own services across VPCs — without using internet gateways, NAT, or VPN. How it works: service providers create VPC Endpoint Services (backed by NLB); consumers create Interface Endpoints (ENIs in their VPC) that receive private IPs. Traffic flows entirely within the AWS network via the Endpoint. Use cases: access SaaS products (Datadog, Snowflake) privately, expose your microservices to other VPCs without full VPC peering, access AWS services (S3, DynamoDB, ECR) without internet. Gateway Endpoints for S3 and DynamoDB are free; Interface Endpoints have hourly and data transfer fees.

Q118. What is AWS Global Accelerator?

Ans: Global Accelerator improves global application performance using AWS's private backbone network. Users connect to the nearest AWS edge location via Anycast IP addresses, then traffic routes to your application endpoint (ALB, NLB, EC2, Elastic IP) over AWS's high-performance network — avoiding congested public internet paths. Benefits: up to 60% improved latency for global users, automatic failover between regions in < 30 seconds, DDoS protection via Shield Standard, and static IP addresses (2 Anycast IPs per accelerator). Compare to CloudFront: CloudFront caches content at edge; Global Accelerator proxies traffic to origin with TCP/UDP support.

Q119. What is AWS VPN?

Ans: AWS provides two VPN options: Site-to-Site VPN (encrypted IPsec tunnel between on-premises network and VPC via Virtual Private Gateway or Transit Gateway — two tunnels for redundancy, up to 1.25 Gbps per tunnel, BGP or static routing), and Client VPN (OpenVPN-based managed service enabling remote users to securely access AWS and on-premises resources — scales to thousands of concurrent users, mutual certificate authentication or Active Directory/SAML). VPN is the low-cost option for internet-based encrypted connectivity. For consistent performance, use Direct Connect; add VPN as backup or encryption layer.

Q120. What is NAT Gateway?

Ans: NAT (Network Address Translation) Gateway allows instances in private subnets to initiate outbound internet connections while blocking inbound internet-initiated connections. Deploy NAT Gateway in a public subnet with an Elastic IP; update private subnet route table to send 0.0.0.0/0 to the NAT Gateway. For high availability, deploy a NAT Gateway in each AZ – route each AZ's private subnets to the local NAT Gateway (avoid cross-AZ data transfer charges). NAT Gateway scales automatically to 45 Gbps. Alternative: NAT Instance (EC2-based, cheaper for low traffic but you manage HA and scaling) – rarely used now.

Q121. What is Route 53 Resolver?

Ans: Route 53 Resolver provides DNS resolution for VPCs. Inbound Endpoints allow on-premises DNS servers to forward queries for AWS resources to Route 53 Resolver. Outbound Endpoints allow VPC resources to forward queries for on-premises domains to on-premises DNS servers – enabling split-horizon DNS. Resolver Rules define forwarding rules (domain → DNS server). DNS Firewall (Route 53 Resolver DNS Firewall) blocks DNS queries to malicious domains – uses AWS managed domain lists (malware, botnet C2 servers) or custom domain lists. Essential for hybrid cloud architectures where on-premises systems need to resolve AWS resources and vice versa.

Q122. Explain VPC Flow Logs.

Ans: VPC Flow Logs capture IP traffic information at the VPC, subnet, or ENI level. Fields: source/destination IP and port, protocol, packets, bytes, action (ACCEPT/REJECT), and more. Use cases: network troubleshooting (why is traffic being rejected), security analysis (detect port scanning, unusual traffic patterns), compliance auditing, and cost optimization (identify cross-AZ data transfer). Flow Logs go to CloudWatch Logs or S3. Query with CloudWatch Logs Insights or Athena (for S3). Enable at VPC level for comprehensive coverage. Custom flow log formats add fields like traffic type, pkt-srcaddr (true source through NAT), and connection tracking state.

Q123. What is AWS Wavelength?

Ans: Wavelength embeds AWS compute and storage at telecom providers' 5G networks, enabling ultra-low latency (single-digit milliseconds) for mobile applications by processing data at the edge near end users — eliminating backhaul to AWS regions. Use cases: real-time gaming, AR/VR, autonomous vehicles, live video streaming, and IoT applications needing < 10ms latency. Wavelength Zones appear as availability zones in your region — you deploy EC2 instances, ECS, and EKS worker nodes into Wavelength Zones. Traffic from 5G devices reaches Wavelength without traversing the internet. Carriers: Verizon, KDDI, SK Telecom, Vodafone.

Serverless Architecture

Q124. What is the serverless architecture pattern?

Ans: Serverless architecture means building and running applications without managing servers. Cloud provider handles provisioning, scaling, patching, and HA. You pay only for actual execution (no idle capacity cost). AWS serverless stack: Lambda (compute), API Gateway (HTTP API), DynamoDB (database), S3 (storage), SQS/SNS/EventBridge (messaging), Cognito (authentication), Step Functions (workflow orchestration), AppSync (GraphQL). Benefits: no server management, automatic scaling to zero, pay-per-use, faster time to market. Challenges: cold starts, execution limits, debugging complexity, vendor lock-in, and stateless design requirements.

Q125. What is AWS API Gateway?

Ans: API Gateway is a fully managed service to create, publish, and manage REST, HTTP, and WebSocket APIs. Types: REST API (feature-rich — request/response transformation, usage plans, API keys, caching), HTTP API (simpler, cheaper, lower latency — for Lambda and HTTP backends), WebSocket API (two-way communication for chat, real-time dashboards). Integrations: Lambda (most common), HTTP backends, AWS services (direct SQS/DynamoDB integration). Features: throttling (requests/second limits), caching (reduce Lambda invocations), authorization (Lambda authorizers, Cognito, IAM, JWT), custom domain names with ACM, and CloudWatch logging.

Q126. What is AWS Step Functions?

Ans: Step Functions is a serverless workflow orchestration service. You define state machines in Amazon States Language (JSON) with states: Task (invoke Lambda, ECS, DynamoDB, SQS, etc.), Choice (conditional branching), Parallel (execute branches concurrently), Map (iterate over array), Wait (pause execution), Succeed/Fail. Express Workflows (high throughput, event-driven – 100,000 executions/second, up to 5 minutes) vs Standard Workflows (long-running, exactly-once semantics – up to 1 year). Use cases: order processing pipelines, ML model training, ETL workflows, microservice orchestration. SDK Integration allows calling 200+ AWS services directly without Lambda.

Q127. What is AWS EventBridge?

Ans: EventBridge is a serverless event bus that connects applications via events. Components: Event Bus (default AWS bus for AWS service events, custom bus for your events, partner event buses for SaaS like Zendesk/Datadog), Rules (event patterns that route matching events to targets), Targets (Lambda, SQS, SNS, Step Functions, API Gateway, Kinesis, EC2, ECS – 20+ targets), Pipes (point-to-point integration with filtering and enrichment), and Schema Registry (auto-discovers event schemas for code generation). Use cases: React to AWS resource changes (EC2 state change), trigger deployments on CodeCommit push, scheduled jobs (cron), and SaaS integration.

Q128. What is AWS Cognito?

Ans: Cognito provides authentication, authorization, and user management for web/mobile apps. Two components: User Pools (user directory – signup/signin, MFA, social identity federation with Google/Facebook/Apple, SAML federation with enterprise IdPs, JWT tokens for API authorization), App Clients (OAuth 2.0 authorization code flow). Identity Pools (Federated Identities – exchange User Pool tokens or social IdP tokens for temporary AWS credentials via STS, enabling direct AWS service access from frontend without backend proxy). Use Cognito User Pools + API Gateway + Lambda + DynamoDB for a fully serverless, auth-protected application.

Q129. What is AWS AppSync?

Ans: AppSync is a managed GraphQL service for building APIs with real-time and offline capabilities. Define a GraphQL schema; AppSync auto-generates resolvers connecting to DynamoDB, Lambda, OpenSearch, Aurora Serverless, HTTP APIs, or custom data sources. Real-time subscriptions push data updates to clients over WebSockets. Offline sync with Amplify DataStore handles conflict resolution when mobile clients reconnect. Authorization modes: API Key (development), IAM, Cognito User Pools, OIDC, Lambda authorizer. Use AppSync when you need flexible queries (request exactly the data you need), aggregating multiple backends in one API, or real-time collaborative features.

Q130. What are Lambda Layers?

Ans: Lambda Layers are ZIP archives containing libraries, runtime dependencies, or custom runtimes that can be shared across multiple Lambda functions. Benefits: reduce deployment package size (layers are stored separately, functions reference them), share common code (utility libraries, security SDKs) across functions, and separate concerns. A function can reference up to 5 layers. Layers are extracted to `/opt` in the execution environment. Public layers (AWS SDK, AWS Data Wrangler) are maintained by AWS. Best practice: put large, stable dependencies (numpy, pandas) in layers; keep business logic in the function package for fast deployments.

Q131. What is the Lambda execution environment lifecycle?

Ans: Lambda execution lifecycle: Init phase (downloads function code and layers, initializes runtime, runs initialization code outside handler — runs once per cold start), Invoke phase (executes handler function), Shutdown phase (runtime gracefully shuts down after period of inactivity). Cold Start: first invocation or after inactivity — Init phase adds latency (100ms–several seconds). Warm start: Lambda reuses existing execution environment — no Init phase, much faster. Mitigation: Provisioned Concurrency (pre-initializes N environments, eliminates cold starts, extra cost), function warming (schedule EventBridge to invoke function every few minutes), and SnapStart (for Java — pre-initialized snapshots).

Q132. What is Lambda@Edge?

Ans: Lambda@Edge runs Lambda functions at CloudFront edge locations globally, triggered by CloudFront events: Viewer Request (before CloudFront checks cache), Origin Request (on cache miss, before request to origin), Origin Response (after origin response, before caching), Viewer Response (before sending to viewer). Use cases: URL rewriting/redirecting, A/B testing (modify request to different origins based on cookie), authentication (verify JWT at edge), image resizing (resize images at edge based on device), header manipulation, and geo-based content serving. Limitations: max 5 seconds (viewer), 30 seconds (origin), Node.js/Python only, 128MB–10GB memory.

Q133. What is AWS SAM (Serverless Application Model)?

Ans: SAM is an open-source framework for building serverless applications. SAM templates extend CloudFormation with shorthand resources: `AWS::Serverless::Function`, `AWS::Serverless::Api`, `AWS::Serverless::SimpleTable` (DynamoDB). SAM CLI: `sam init` (project templates), `sam build` (packages dependencies), `sam local invoke` (test Lambda locally with Docker), `sam local start-api` (local API Gateway), `sam deploy` (deploys via CloudFormation). SAM Accelerate (`sam sync`) enables rapid local code changes pushed to cloud in seconds. SAM integrates with CodePipeline for CI/CD. Alternative: Serverless Framework (community-led, multi-cloud).

Database & Storage Services

Q134. What is Amazon Aurora and its advantages?

Ans: Aurora is AWS's cloud-native relational database — MySQL and PostgreSQL compatible, up to 5x faster than MySQL and 3x faster than PostgreSQL at 1/10th the cost of commercial databases. Architecture: storage (distributed across 6 copies in 3 AZs, auto-healing, grows in 10GB increments up to 128TB) is separate from compute (instances). Features: Aurora Serverless v2 (scales capacity in fractions of ACUs, from 0 to 128 ACUs in seconds), Global Database (cross-region replication < 1 second RPO, < 1 minute RTO), parallel query, Babelfish (SQL Server T-SQL compatibility). Use Aurora for new relational workloads requiring HA.

Q135. What is DynamoDB Accelerator (DAX)?

Ans: DAX is an in-memory cache for DynamoDB — fully compatible with DynamoDB APIs, providing microsecond read latency vs millisecond for DynamoDB. DAX is a write-through cache: writes go to DAX and DynamoDB simultaneously, reads return from cache if available. DAX clusters are multi-AZ (3–11 nodes). Use DAX for: high-read workloads (product catalogs, gaming leaderboards, social feeds), applications with repeated reads of the same items, and cost reduction by reducing DynamoDB read capacity units. DAX is NOT suitable for strongly consistent reads, write-heavy workloads, or applications that rarely read the same item twice.

Q136. What is Amazon ElastiCache?

Ans: ElastiCache is a managed in-memory caching service. Two engines: Redis (rich data structures — strings, lists, sets, sorted sets, hashes; persistence, pub/sub, Lua scripting, Cluster Mode for horizontal scaling to 500+ nodes, replicas for HA, Global Datastore for cross-region replication) and Memcached (simpler, multi-threaded, horizontally scalable, no persistence). Use cases: database query caching (reduce RDS load), session storage (stateless application sessions), leaderboards (Redis sorted sets), rate limiting, pub/sub messaging. ElastiCache Serverless (new) scales capacity automatically.

Q137. What is Amazon Redshift?

Ans: Redshift is a fully managed, petabyte-scale cloud data warehouse. Columnar storage, massively parallel processing (MPP), and compression achieve fast query performance on large datasets. AQUA (Advanced Query Accelerator) hardware-accelerates queries. Redshift Spectrum queries S3 data lake directly without loading data. Redshift Serverless scales automatically based on workload. Integration with: S3 (COPY and UNLOAD commands), Kinesis Firehose, Glue (ETL), QuickSight (visualization), and SageMaker. Use Redshift for OLAP workloads — reporting, BI, complex analytics on historical data. Not for OLTP (use RDS/Aurora).

Q138. What is Amazon Athena?

Ans: Athena is a serverless, interactive query service for analyzing data in S3 using standard SQL. No infrastructure to manage — pay per query (\$5 per TB scanned, reduced by compression and partitioning). Supported formats: CSV, JSON, Parquet, ORC, Avro, and more. Use Parquet/ORC with Snappy compression to reduce costs 10–100x. Partitioning (e.g., by year/month/day) skips irrelevant data. Athena Federated Query accesses data in RDS, DynamoDB, Redshift, CloudWatch Logs, and custom sources. Athena for Apache Spark runs Spark workloads. Connect to QuickSight for visualization. Ideal for log analysis, ad-hoc queries on data lake.

Q139. What is AWS Glue?

Ans: Glue is a fully managed ETL (Extract, Transform, Load) service. Components: Data Catalog (centralized metadata repository — tables, schemas, partitions — shared by Athena, Redshift Spectrum, EMR), Crawlers (automatically infer schema from S3, RDS, DynamoDB, JDBC sources), ETL Jobs (PySpark or Scala Spark for data transformation, generated or custom code), Triggers (schedule or event-based job execution), and Glue DataBrew (visual, no-code data preparation). Glue Studio provides visual ETL job designer. Use Glue for data lake ETL pipelines, data catalog management, and integrating diverse data sources.

Q140. What is Amazon S3 and its key features?

Ans: S3 (Simple Storage Service) is AWS's infinitely scalable object storage. Key features: 11 nines (99.999999999%) durability (data replicated across 3+ AZs), versioning (retain all versions of objects), lifecycle policies (auto-transition/delete), Server-Side Encryption (SSE-S3, SSE-KMS, SSE-C), S3 Object Lock (WORM — write once read many, for compliance), Replication (cross-region or same-region), Event Notifications (S3 → SQS/SNS/Lambda on object create/delete), S3 Select (SQL query on individual objects), Intelligent-Tiering (auto cost optimization), and Access Logs. S3 is used for backups, static websites, data lakes, and artifact storage.

Q141. What is the difference between S3 and EFS?

Ans: S3 is object storage — accessed via HTTP API (GetObject, PutObject), ideal for large files stored and retrieved occasionally, supports any file size, globally accessible, and highly cost-effective. EFS is a file system — mounted via NFS protocol, accessed like a traditional filesystem with file paths, directories, and POSIX permissions. EFS supports concurrent access from multiple EC2 instances simultaneously — S3 doesn't support real-time concurrent writes. S3 has millisecond latency for HTTP; EFS has sub-millisecond latency for NFS. Use S3 for data lakes and static assets; use EFS for shared configuration files, content management, and machine learning training data.

Q142. What is Amazon FSx?

Ans: FSx provides fully managed, high-performance file systems. FSx for Windows File Server: SMB protocol, Active Directory integration, Windows NTFS — for Windows workloads, .NET apps, SQL Server, home directories. FSx for Lustre: parallel file system for HPC, ML training, video processing — integrates with S3 as a data repository, sub-millisecond latency, hundreds of GB/s throughput. FSx for NetApp ONTAP: multi-protocol (NFS, SMB, iSCSI), data management features (snapshots, cloning, tiering), SnapMirror for cross-region replication. FSx for OpenZFS: Linux-based workloads, NFS. Use FSx when workloads require specific filesystem protocols beyond EFS.

Q143. What is AWS Database Migration Service (DMS)?

Ans: DMS migrates databases to AWS with minimal downtime. Supports: homogeneous migrations (MySQL → RDS MySQL) and heterogeneous migrations (Oracle → Aurora, SQL Server → PostgreSQL). How it works: create replication instance (EC2 for migration compute), define source and target endpoints, create task (Full Load, CDC — Change Data Capture, or both). Full Load + CDC: copies existing data then replicates ongoing changes, allowing you to keep source running until cutover. AWS Schema Conversion Tool (SCT) converts schemas and stored procedures between database engines. DMS is also used for continuous replication to data warehouses and Kafka.

Advanced DevOps Practices

Q144. What is Site Reliability Engineering (SRE)?

Ans: SRE applies software engineering principles to operations problems. Key concepts: SLIs/SLOs/SLAs (measuring reliability), Error Budgets (acceptable downtime per quarter — if budget exhausted, freeze feature work and focus on reliability), Toil Elimination (automating repetitive operational work), Blameless Post-Mortems (learn from incidents without blame), On-Call Rotations (equitable sharing of on-call burden). On AWS: implement SRE practices with CloudWatch (SLI measurement), dashboards (error budget tracking), runbooks in OpsCenter or Confluence, PagerDuty/OpsGenie for on-call management, and automated remediation via EventBridge + Lambda.

Q145. What is Chaos Engineering?

Ans: Chaos Engineering deliberately injects failures into systems to test resilience and identify weaknesses before they cause real outages — 'breaking things on purpose to learn'. Principles: define steady state, form a hypothesis, run experiments in production (or staging), minimize blast radius, automate experiments. AWS tools: AWS Fault Injection Simulator (FIS) — terminates EC2 instances, throttles DynamoDB, injects network latency, simulates AZ failures. Community tools: Chaos Monkey (terminates random EC2), Gremlin (comprehensive SaaS platform). Game Days: scheduled chaos events where teams run experiments and practice incident response. Start small, increase complexity as confidence grows.

Q146. What are the 12-Factor App principles?

Ans: The 12 Factors are best practices for building cloud-native, scalable applications: 1) Codebase (one codebase in VCS), 2) Dependencies (explicitly declare all dependencies), 3) Config (store in environment variables, not code), 4) Backing services (treat as attached resources), 5) Build/Release/Run (strict separation), 6) Processes (stateless, share-nothing), 7) Port binding (self-contained, export via port), 8) Concurrency (scale via process model), 9) Disposability (fast startup, graceful shutdown), 10) Dev/prod parity (keep environments similar), 11) Logs (treat as event streams), 12) Admin processes (run as one-off processes). AWS Lambda and ECS enforce many of these naturally.

Q147. What is GitFlow branching strategy?

Ans: GitFlow is a branching model with main, develop, feature, release, and hotfix branches. Main contains production code; develop is integration branch. Features branch from develop (feature/login), merged back via PR. Release branches (release/1.0) branch from develop, receive only bug fixes, merged to both main and develop. Hotfix branches (hotfix/security-patch) branch from main for urgent production fixes, merged to both main and develop. GitFlow suits release-based delivery with scheduled releases. For continuous delivery, Trunk-Based Development (TBD) is preferred – developers commit to main/trunk daily using feature flags, with short-lived feature branches < 1 day.

Q148. What is feature flag management?

Ans: Feature flags (feature toggles) enable/disable features at runtime without code deployment – decoupling deployment from release. Types: Release flags (dark launch new features), Experiment flags (A/B testing), Ops flags (kill switches for broken features), Permission flags (gradual rollout to specific users). AWS implementation: AppConfig (part of SSM) provides dynamic configuration with validation, deployment strategies, and rollback. Third-party tools: LaunchDarkly, Split.io, Unleash. Best practices: clean up old flags (flag debt), use structured naming conventions, audit who can change flags, and monitor flag state in dashboards. Feature flags enable continuous delivery without risking full exposure.

Q149. What is a microservices architecture?

Ans: Microservices decompose applications into small, independently deployable services – each owns its data and communicates via APIs. Principles: single responsibility (each service does one thing well), loose coupling (services don't share databases), high cohesion, independently deployable. AWS microservices stack: API Gateway (ingress), ECS/EKS (container orchestration), ALB (service routing), App Mesh (service-to-service communication), SQS/EventBridge (async communication), DynamoDB/Aurora (separate databases per service), X-Ray (distributed tracing). Challenges: distributed transactions, network latency, operational complexity, service discovery. Use microservices for teams that can own independent services end-to-end.

Q150. What is the Strangler Fig pattern?

Ans: The Strangler Fig pattern is a migration strategy for gradually replacing a monolith with microservices. Named after the strangler fig tree that grows around a host tree until it replaces it. How it works: deploy new microservice alongside monolith □ route a subset of requests to the new service (using API Gateway or feature flags) □ gradually increase traffic □ eventually retire the monolith function. AWS implementation: API Gateway in front of monolith routes /new-feature to Lambda/ECS while /legacy routes to existing EC2. Benefits: incremental migration (reduces risk), continuous delivery during migration, no big-bang rewrite. The monolith 'strangles' over months or years.

Q151. What is DevSecOps?

Ans: DevSecOps integrates security into every phase of DevOps — shifting security left and making it everyone's responsibility. Practices: SAST (Static Application Security Testing — CodeGuru Reviewer, SonarQube scan code in CI), DAST (Dynamic Application Security Testing — OWASP ZAP against running application in staging), SCA (Software Composition Analysis — Snyk, OWASP Dependency-Check for vulnerable libraries), IaC Scanning (Checkov, cfn-nag for CloudFormation, tfsec for Terraform), Container Scanning (ECR Image Scanning, Trivy), Secrets Detection (git-secrets, truffleHog in pre-commit hooks). Security gates in CodePipeline fail builds on critical findings.

Q152. What is a post-mortem and how do you run one?

Ans: A post-mortem (or incident review) is a structured analysis of an incident to understand what happened and prevent recurrence. Blameless culture: focus on system failures, not human error. Structure: Incident Summary (date, duration, impact, severity), Timeline (chronological events — what happened and when), Root Cause Analysis (5 Whys, fishbone diagram), Impact Assessment (users affected, revenue impact, SLO breach), What Went Well (detection speed, communication), Areas for Improvement, Action Items (specific, assigned, time-bound). AWS tools: CloudTrail (audit trail), CloudWatch Logs (application events), X-Ray (distributed trace). Share post-mortems broadly for organizational learning.

Q153. What is Platform Engineering?

Ans: Platform Engineering builds Internal Developer Platforms (IDPs) — self-service infrastructure and tooling that enables product teams to deploy and operate applications without deep infrastructure expertise. The platform team provides: golden paths (opinionated, pre-approved ways to deploy), developer portals (Backstage), self-service CI/CD templates (CodePipeline/GitHub Actions workflows), infrastructure blueprints (CloudFormation/CDK constructs), observability tooling, and developer environments. AWS tools: Control Tower (account vending), Service Catalog (self-service infrastructure), Proton (environment templates). Platform engineering reduces cognitive load on developers while maintaining organizational standards for security and compliance.

Q154. What is GitOps with ArgoCD on EKS?

Ans: ArgoCD is a GitOps continuous delivery tool for Kubernetes — it syncs the cluster state with Git repositories. Workflow: developers push Kubernetes manifests (or Helm/Kustomize) to Git □ ArgoCD detects the change □ compares desired state (Git) to actual state (EKS) □ automatically applies differences. ArgoCD provides: multi-cluster management, application health monitoring, sync status, rollback (git revert □ ArgoCD resync), RBAC for team access, and webhook triggers for immediate sync on push. On EKS: deploy ArgoCD with Helm, configure IAM roles for EKS access, set up OIDC authentication with AWS SSO. ArgoCD + EKS + ECR = modern GitOps pipeline for Kubernetes.

Q155. What is AWS Backup?

Ans: AWS Backup is a centralized, managed backup service across AWS resources — EC2 AMIs, EBS, RDS, DynamoDB, EFS, FSx, Aurora, Storage Gateway, S3, VMware on-premises. Create Backup Plans with rules defining frequency (hourly to yearly), retention, lifecycle (transition to cold storage after X days), and target vault. Backup Policies via Organizations enforce backup rules across all accounts. Cross-region and cross-account copy for DR. Vault Lock (WORM) prevents backup deletion for compliance. Restore testing (automated restore + validation) verifies backup integrity. AWS Backup replaces custom backup scripts and provides unified compliance reporting for auditors.

Q156. What is the principle of least privilege and how do you implement it?

Ans: Least privilege means granting only the minimum permissions required for a task – nothing more. Implementation: start with AWS-managed policies for known use cases, scope resource ARNs specifically (not wildcards), use IAM conditions (aws:RequestedRegion, aws:PrincipalTag, aws:SourceVpc), use permission boundaries to limit developer IAM permissions, regularly review with IAM Access Analyzer's unused access feature, use SCPs to prevent privilege escalation at the org level. For applications: EC2 instance profiles and ECS/EKS task roles with specific DynamoDB table ARNs rather than full DynamoDB access. Audit quarterly and remove permissions that haven't been used in 90 days.

Q157. What is AWS Well-Architected Tool?

Ans: The Well-Architected Tool guides architecture reviews against AWS best practices. You create a workload (name, environment, AWS regions), answer questions across the six pillars (Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, Sustainability), and it identifies High/Medium/Low risk issues and generates an improvement plan. Custom lenses support industry-specific (Financial Services, Healthcare, SaaS) or organizational standards. Milestone snapshots track progress over time. Use it: before launching new workloads, during quarterly architecture reviews, for regulatory compliance evidence, and onboarding new teams to AWS best practices. Free to use.